

Vesuvius Challenge 2位解法 — post-processingの勝利

Rank	2
Team	risk of overfitting
Author	Marius Heuser
Topic ID	679278
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679278>
Retrieved with Kaggle CLI on 2026-07-07.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

Kaggleと主催者、特にホストのGiorgio AngelottiとSean Johnsonに感謝する。難しい判断も含め継続的なcommunicationが素晴らしかった。

概要

nnUNetで $128^3 \cdot 160^3$ patch modelを訓練し、 160^3 を60%、 128^3 を40%でlogit ensemble。post-processingは複数threshold maskを使うlocal component-wise interpolationである。

Model

vanilla nnUNet ResEncTrainer で以下を訓練した。

```
patch size: 128 / batch size: 2 / epochs: 1000 / fold: all
patch size: 160 / batch size: 2 / epochs: 1000 / fold: all
```

当初はtrainの80%で 128^3 modelを訓練し、そのthresholdを全データ 128^3 へ転用できた。 160^3 は終盤に完成しthreshold tuning時間がなく、visual確認と少数LB提出でensemble設定を決めた。

loss改善へ多くの時間を使い、Skeaw-BorT Lossはbackgroundをcomponentへ分割できず不適合、medial-surface lossはTopoを上げる一方Surface Diceを落とした。Topoはpost-processingで改善できるため、modelはSurface Dice優先がよかった。

Post-processing

logit ensemble (mirror+rotation TTA) から複数maskを作った。各maskは3D hysteresis、anisotropic closing、dust removalを行い、volume各面の3 voxels (label 2) を0にし、再度size 1000のdust removal。baseは $T_{low}=0.2$ 、refinement用は0.5、0.6、0.7。baseへbinary_fill_holesし、各面3 voxelsをcropした。

Euler numberからtunnel/holeを高速検出し、次のlocal interpolationを行った。

1. base segmentationをsingle-component volumeへ分割。
2. overlap 50%のsmall windowごとにEuler numberでtunnel/holeを検査。
3. Euler ≥ 1 ならcomponentを保存。Euler < 1 のpatchは保存し、overlap patchをmerge。
4. 各patchを座標補間で2D gridへ投影し、smooth、edge flood-removeでover-interpolationを除去、3-voxelへdilationしてbinary_fill_holes。
5. 新旧componentのDiceとcoverageが十分ならpatch内部を置換。
6. 不十分ならより高threshold segmentationの各componentで繰り返し、sheet mergeを解く。
7. 条件を満たすか削除されるまで繰り返す。

最後にremove_small_objectsし元sizeへpadした。thresholdが上がるにつれmerged sheetが分離する性質を利用した。

最終手法は締切1日前に実装。以前のglobal interpolationはPrivate 0.631だがPublic/CVで悪く、label 2領域へsheetが入りやすかった。local版はtuning不足ながらより優れていた。

- [Global interpolation notebook \(Private 0.631\)](#)
- [Final local interpolation notebook \(Private 0.622\)](#)

Claudeによる最適化でvolume当たり約3分から約1分へ短縮した。公開codeはcomponent全体のEuler numberを一度に調べ、hole/tunnelなら完全置換する点など本文版と差がある。

補足Q&A

pp1985: これらのpost-processing案をどう発見しましたか。visualizationが大きかったですか？

Marius Heuser: visualizationは非常に役立ったし、時間も重要だった。3〜4回行き詰まったが、別のことを試して戻ることによって各問題の解法を見つけた。説明も追記した。

Arpit1Bansal: なぜ1000 epochs必要と判断しましたか？

Duong Nguyen (teammate) : 1000 epochsはnnU-Net default。最初はbaselineとしてdefaultを使ったが訓練が長かったため、それ以上は増やさなかった。